OPERATION KIVOTOS · UNITY 문제해결 (1)

큰 맵에서의 몬스터 관리 5가지 방법 비교 계획서

같은 문제에 대한 5가지 방법을 전부 구현해서 같은 기준으로 재고,
그 결과로 이 게임의 조건에 맞는 조합을 고른다.

수록 문서

1. **README** — 요약 · 서사 골격 · 확정된 결정
2. **01 Methods** — 5가지 방법 정의, 분류명, 구현 판단, 채택 조건
3. **02 Measurement** — 하네스, 시나리오 3종, N 3단계, 지표, 절차
4. **03 Predictions** — 측정 전 예측 박제 (수정 금지)
5. **04 Comparison** — 비교표 골격 + 정성 비교 (현재는 예상)
6. **05 PortfolioNarrative** — 구성, 비중, 함정, 덱 안에서의 위치
7. **06 References** — 참고 자료와 인용 시 주의

작성 2026-08-19 · docs/MonsterSimulation/Comparison/
도식 중심 설명은 별도 파일 `MonsterManagement_5Methods.pdf` 참조

몬스터 관리 방식 비교 — 5가지 방법

> 작성: 2026-08-19 · 대상: 포트폴리오 Unity 파트 **문제해결(1)** 재설계 상위 폴더 MonsterSimulation/의 **비교 실험 트랙**. 01~05가 "무엇을 만들까"라면 여기는 **"여러 방법을 만들어서 재고, 그걸로 무엇을 주장할까"** 를 다룬다. **이 README만 읽어도 맥락 없이 재개할 수 있도록** 결론을 요약해 둔다.

한 줄 요약

같은 문제("큰 맵에서 누구를 켜둘 것인가")에 대한 **5가지 방법을 전부 구현해서 같은 기준으로 재고**, 그 결과로 "이 게임의 조건에서는 이 조합"이라는 결론을 낸다. 파는 것은 기법이 아니라 **선택의 근거**다.

서사 (포트폴리오 슬라이드 골격)

```
출발점    방법2 (단일 Sector)  ← 내가 만들어 굴리던 것. 여기서 문제 발견
   ↓
시도 1    방법3  섹터 유지하고 예외 처리   → 버그는 고침, 성능은 그대로
시도 2    방법4  틀을 바꿔 거리 기반       → 규모 해결, 밀집 취약
시도 3    방법5  끄지 말고 느리게          → 밀집 해결, 규모 취약
   ↓
결론      게임 조건에 대입 → 4와 5를 층으로 나눠 겹침
   ↓
마무리    껐는데 메모리는 안 줄어 있었다 → 리소스 수명(Addressable) 파트로 연결

참조선    방법1 (전체 활성화)  ← 서사에는 안 나오고, 측정 표의 상한선으로만 등장
```

핵심 메시지: "증상을 막는 것"과 "원인을 없애는 것"은 다르다. 그리고 **방법3은 일부러 실패하는 단계**다 — 섹터라는 틀 안에서는 정확성과 성능을 동시에 못 잡는다는 것을 숫자로 보여주는 게 그 방법의 역할이고, 그래야 방법4가 "하나 더 해봤다"가 아니라 필연이 된다.

문서 목록

문서	내용
01_Methods.md	5가지 방법 정의 — 정식 분류명, 동작, 구현 판단, 채택 조건, 현재 상태
02_Measurement.md	측정 기준 — 하네스, 시나리오 3종, N 3단계, 지표, 절차
03_Predictions.md	**측정 전 예측 박제.** 재기 전에 써두고 나중에 맞았는지 대조한다
04_Comparison.md	비교표 골격 (측정 후 채울 빈칸) + 정성 비교 (현재는 예상)
05_PortfolioNarrative.md	포폴 구성 — 비중 배분, 이론/구현의 분리, 함정, 덱 안에서의 위치
06_References.md	참고 자료와 인용 시 주의사항
MonsterManagement_5Methods.pdf	도식 중심 쉬운 설명 (8쪽) — 본인 정리용 / 비개발자 설명용
MonsterSimulation_Plan.pdf	위 md 문서 전체를 한 파일로 묶은 계획서

확정된 결정

- **서사의 출발점은 방법2(단일 Sector)** — 방법1(전체 활성화)은 내가 설계한 게 아니라 "아직 아무것도 안 한 상태"라, Before로 쓰면 인위적이다 (01_Problem 3.2 참고). 표에는 상한선으로 남긴다
- **5가지 전부 구현해서 잰다** — "비교했다"고 쓴 것 중 실제로 안 만든 게 있으면 질문 한 번에 무너진다
- **인접 섹터 켜기는 구현하지 않는다** — 화면 25m인데 섹터 100m라 낭비가 2~3배로 늘 뿐. 비교표 행이 아니라 "검토 후 기각" 각주로 처리
- **결론은 하나를 고르는 게 아니라 두 층을 겹치는 것** — 방법4로 존재를, 방법5로 품질을 정한다. 둘은 축이 직교해서 경쟁 관계가 아니다
- **끝은 성능이 아니라 메모리로 닫는다** — G4(껐는데 메모리가 안 줄었다) → Addressable 파트로 연결
- **평균 FPS는 지표로 쓰지 않는다** — p99 / 1% low / 히칭 프레임 수만

지금 당장 해야 할 일 (기록 소실 방지)

> ⚠ **고치기 전에 "경계에서 전투 중 몬스터가 사라지는" Before 영상을 찍어둘 것.** 방법3을 만들고 나면 다시 촬영할 수 없다. 서사에서 방법2의 유일한 시각 증거다.

이후 순서: **측정 하네스 → 방법3 → 방법4 → 방법5 → 45회 측정 → 표 채우기 → 메모리 마무리**

관련 문서

- ../README.md — 상위 설계 트랙 요약
- ../01_Problem.md — 현재 구조 진단. G1(경계 소실) · G2(판정 단위) · G4(메모리) 근거
- ../03_Design.md — A층(존재)/B층(품질) 분리 원칙. 본 비교의 방법4/방법5에 대응
- ../05_Roadmap.md — 단계별 일정. 본 문서의 측정 계획이 S0를 확장한 것
- ../../ResourceSystem/ — 측정 규율(변수 통제·개발빌드 계측)의 선례

01. 5가지 방법 정의

0. 전제 — 게임플레이 방식을 하나로 고정한다

항목	값
맵	끊기지 않는 **하나의 연속된 야외 필드** (로스트아크형)
몬스터	스포너가 맵 곳곳에 산재. 전부 켜면 무겁다
시야	타이트 쿼터뷰. 화면에 들어오는 지면 **약 25m** (실측: `CameraController._delta = (0,4,-5)`)
현재 규모	Sector 5개 / MonsterSpawner 23개 (`BackStreetMap.prefab`)

전제를 고정하면 "실내 던전이면 어떤가" 같은 곁가지가 잘리고, 5가지가 **같은 조건에서 정면으로** 붙는다. (조건이 바뀌면 답이 바뀐다는 이야기는 결론부에서 한 번만 다룬다. → 05_PortfolioNarrative)

0.1 5가지는 임의의 후보가 아니다 — 한 축 위의 네 점 + 직교하는 하나

```
[존재 축] 누구를 존재하게 할 것인가 — 판정 단위가 화면 크기까지 내려온다

방법1          방법2          방법3          방법4
맵 전체   →   구역 100m   →   구역 + 예외   →   반경 약 45m
 (∞)                                            ↑ 화면 지면 ~25m

[품질 축] 존재하는 것을 얼마나 자주 돌릴 것인가  ← 직교
방법5   끄지 않고 틱 등급만 강등
```

방법1~4는 전부 **"누가 존재하는가"** 를 정하는 방식이고 다른 건 판정 단위 하나뿐이다. 방법5만 축이 다르므로 **방법4와 동시에 적용할 수 있다.** 이게 결론의 뼈대다.

방법 1 — 전부 켜두기

분류: 없음 (관리 부재) · **상태**: 이미 있음 (`BenchmarkTester` F1) · **서사 내 위치**: 표의 상한선

동작

아무것도 하지 않는다. 시작 시 모든 스포너가 몬스터를 뽑고, 그 몬스터들은 종료까지 계속 돈다.

좋은 점

- 코드가 0줄. 버그가 생길 구석이 없다
- 몬스터가 사라지거나 튀어나오는 일이 절대 없다
- **몬스터가 수십 마리 규모면 이게 정답이다** — 관리 비용이 이득보다 크다

나쁜 점

- 비용이 전체 몬스터 수 N에 그대로 비례한다

- 화면 반대편 몬스터도 전부 계산한다. 순수 낭비
- 메모리를 최대로 쓴다

주의

현재 포트폴리오의 Before 수치가 이것(F1)인데, **아무도 이 구성으로 출시하지 않으므로 Before로는 약하다.** 그래서 서사의 출발점에서 빼고 표의 상한선으로만 남긴다.

방법 2 — 구역 하나만 켜두기

분류: **Zone-based interest management** · **상태**: 구현되어 있음 (현재 방식) · **서사 내 위치**: 출발점

동작

맵에 트리거 볼륨(Sector)을 5개 놓는다. 플레이어가 진입하면 그 구역 스포너를 전부 `SpawnMonsters()`, 이전 구역은 전부 `DespawnMonsters()`.

실물: Sector.cs · SectorManager.cs · MonsterSpawner.cs

좋은 점

- 단번에 크게 가벼워진다
- 구현이 쉽다. 볼륨 놓고 진입만 감지하면 된다

나쁜 점

- **G1 — 경계에서 때리던 몬스터가 사라진다.** 구역을 넘는 순간 이전 구역이 통째로 꺼진다. `Sector.OnTriggerExit`은 **빈 몸통**이다
- **G3 — 진입 프레임에 스폰이 몰린다.** `spawnList`를 순회하며 모든 UniTask를 즉시 `Forget()`. 프레임 예산이 없다
- **G2 — 판정 단위가 화면보다 크다.** 구역 100m vs 화면 25m. 지금도 화면 밖을 대량으로 켜고 있다
- **G4 — 메모리는 안 준다.** 디스폰 = `Pool.Push`. 풀이 `AcquirePoolRef`로 프리팹 핸들 refCount 티켓을 계속 쥔다
- 맵이 커지면 구역도 커져서 다시 무거워진다. **N에 대한 기울기는 방법1과 같고 상수배만 나아진 것**

수치 주의

"5구역이니 5분의 1"은 **틀리다.** 몬스터가 구역마다 고르게 있지 않다. 기존 실측은 F1 대비 **▼57%** 이므로, 문서·슬라이드에는 실측값을 쓴다.

방법 3 — 구역을 켜되, 싸우는 대상은 예외

분류: Hybrid (zone + 참조 카운팅) · **상태**: 미구현 · **서사 내 위치**: 전환점 (일부러 실패하는 단계)

동작

```
몬스터가 전투 시작 → 소유 스포너에 "사용 중" 등록
스포너: 사용 중인 몬스터가 하나라도 있으면 비활성화 거부
플레이어가 멀어짐 → 몬스터가 원래 위치로 복귀(리쉬) → 등록 해제
사용 중 0 + 플레이어 없음 → 그때 비활성화
```

채택 조건 3가지 (없으면 사고 난다)

1. **카운터(int)가 아니라 집합(HashSet)** + 주기적으로 죽은/null 항목 청소 → 통보를 한 번 놓치면 조용히 새는 구조. HP바 누수 사건과 동일 계열
2. **유예 최대 시간(예: 20초) 강제** — 상한이 없으면 경계에 붙어 계속 싸울 때 **방법1로 수렴한다**
3. **유지 단위를 섹터가 아니라 스포너로** — 한 마리 때문에 구역 전체가 켜져 있을 이유가 없다

좋은 점

- **G1(경계 소실)이 완전히 사라진다.** 이게 목적이다
- 기존 구역 구조를 건드리지 않아도 된다
- 여기서 만드는 **리쉬 복귀는 방법4에서도 그대로 쓴다.** 버리는 작업이 아니다

나쁜 점

- **성능이 좋아지지 않는다.** 두 구역이 동시에 켜져 있는 시간이 생기므로 평균 활성 수가 늘어난다
- G2(낭비) · G3(진입 히치) · G4(메모리)는 하나도 안 고쳐진다
- 최악의 경우(경계에서 계속 전투) **방법1로 수렴한다**

표현 주의 ★

"성능이 **나빠진다**"고 단정하지 말 것. 경계 전투는 상시가 아니라서 **실측은 방법2와 비슷하게 나올 수 있다.** 서사가 "3번이 성능을 되돌려놓는다"에 걸리면 측정 결과에 따라 무너진다. 측정과 무관하게 확실히 참인 것만 주장한다 → **"버그는 고쳤지만 성능은 조금도 좋아지지 않았고, 최악의 경우 방법1로 수렴한다."** 실제로 나빠지면 그때 수치를 근거로 강하게 쓰면 된다.

방법 4 — 내 주변 거리로만 판단

분류: **Aura-based interest management** · **상태**: 미구현 · **서사 내 위치**: 본론

동작

```
group.SetBoundingSpheres(spawnerSpheres);         // 로드 시 1회. 스포너는 안 움직임
group.SetBoundingDistances(new[]{ 25f, 35f, 50f });
group.SetDistanceReferencePoint(playerTransform); // 카메라가 아니라 플레이어
group.onStateChanged += OnBandChanged;            // 폴링 없음
```

구역·경계·인접이라는 개념 자체가 사라진다. 기하학에서 저절로 나온다.

구현 판단 4가지 — 슬라이드에 들어갈 알맹이는 이론이 아니라 여기다

판단	이유
몬스터가 아니라 **스포너**를 등록	스포너는 안 움직이므로 스피어 갱신 비용 0. 몬스터를 등록하면 매 프레임 위치를 다시 써야 함
기준점 = 카메라가 아니라 **플레이어**	카메라 회전만으로 세계가 스폰/디스폰되면 안 됨
밴드 경계에 **히스테리시스**	안 넣으면 경계선 위에서 왕복할 때 깜빡임. 방법3이 고쳤던 문제가 다른 얼굴로 돌아옴
디스폰돼도 **리스폰 타이머는 스포너가 소유**	화면 밖 세계가 멈추지 않음. 대표값 시뮬레이션

좋은 점

- **경계가 존재하지 않으므로 경계 버그도 없다.** 고친 게 아니라 없앤 것
- 화면에 보이는 만큼만 켠다. 낭비 최소
- **맵이 커져도 활성 개체 수가 늘지 않는다** (밀도가 일정하다는 전제). 방법2가 상수배 개선인 데 반해 **점근적 개선**
- 안 쓰는 몬스터를 실제로 지우므로 **메모리를 줄일 여지가 생긴다**

나쁜 점

- 팝인 — 멀리 있던 몬스터가 밴드에 들어오면 없다가 생긴다
- 밴드 경계 깜빡임 (히스테리시스로 막는다)
- **밀집에 무력하다.** 몬스터가 한곳에 몰리면 전부 밴드 안이라 걸러낼 게 없다 → 방법5가 필요한 이유

메모리 조건 ★

방법4만 해서는 메모리가 줄지 않는다. 디스폰이 `Pool.Push`에서 끝나고 풀이 프리팹 핸들 티켓을 계속 쥐고 있기 때문(G4). **풀·핸들 반납 정책을 같이 고쳐야** 실제 MB가 떨어진다. 이 조건을 명시하는 것이 오히려 이득이다 — Addressable 리소스 파트로 연결되는 고리가 여기서 생긴다.

완전 상수가 아니라는 단서

CullingGroup도 스포너 수만큼 스피어를 돈다. 다만 개체당 AI 비용보다 2~3자릿수 싸서 곡선이 평평해 보일 뿐이다. 이 단서를 직접 다는 편이 신뢰를 준다.

방법 5 — 끄지 않고, 대신 느리게

분류: **Significance / LOD 기반** (UE `Significance Manager`, `Mass AI` 계열) · **상태**: 미구현 · **서사 내 위치**: 축이 다른 보완재

동작

```
60Hz 버킷 : 전투 중 / 화면 안
10Hz 버킷 : 근처, 비전투
 2Hz 버킷 : 멀리 (존재만)
```

- 같은 버킷 안에서도 **위상 오프셋**으로 프레임을 어긋내 스파이크 방지
- **"생각은 가끔, 움직임은 매 프레임."** 목적지 결정만 저빈도로 하고 이동·회전은 매 프레임 보간 → 등급 전환이 눈에 안 보인다
- **승격은 즉시, 강등만 유예** — 피격·어그로는 즉시 최고 등급으로

전제 작업

몬스터가 저마다 `Update()`에서 `_topNode.Evaluate()`를 도는 구조를 먼저 걷어내야 한다 (BaseMonsterController.cs:99). = 시뮬레이션 매니저(제어점) 확보. 등록/해제 대칭 + 주기적 청소 + 제거 대기 큐 + **경과 시간 주입**.

좋은 점

- **팝인이 원천적으로 없다.** 아무도 사라지거나 생기지 않는다
- **밀집에서 유일하게 효과가 있다.** 다 가까이 있어도 등급을 나눠 깎을 수 있다
- 방법4와 축이 직교하므로 **같이 쓸 수 있다**
- 업계 주류 방향과 일치한다 (→ 06_References)

나쁜 점

- **메모리가 1바이트도 안 준다.** 아무도 지우지 않으므로
- **규모에 약하다.** 맵이 10배가 되면 2Hz 버킷도 10배가 되고 오브젝트도 10배 그대로다. 방법1보다 훨씬 완만할 뿐 **여전히 N에 비례한다**
- 멀리 있는 개체의 반응이 둔해진다

범위를 좁게 자를 것

틱 버킷 3개 + 위상 오프셋까지만. 중요도 점수·타입별 예산(Significance Manager 급)은 손대지 말고 "다음 단계"로 남긴다. 착수 판단 근거는 방법4 실측에서 나오는 **35m 내 몬스터 수 분포**.

검토 후 기각 — 인접 섹터도 함께 켜기

방법2를 보면 가장 먼저 떠오르는 개선이지만 **구현하지 않고 각주로만 남긴다.**

1. **데이터가 썩는다** — 섹터를 옮기거나 추가할 때마다 이웃 목록을 손봐야 하고, 안 고치면 조용히 틀린다
2. **단위 크기가 안 맞는다** — 화면 25m인데 섹터 100m. 인접까지 켜면 낭비가 2~3배

구현해서 재는 비용 대비 결론이 뻔하다. 다만 **"검토했고 왜 버렸는지"는 반드시 남긴다** — 없으면 "생각 못 했나"로 읽히고, 있으면 "따져보고 버렸구나"가 된다.

상태 요약

방법	분류	상태	예상 작업량
1 전부 켜기	—	✅ 있음 (F1)	0
2 구역	Zone-based	✅ 있음 (현재)	0
3 구역+예외	Hybrid	⬜ 미구현	3~4일
4 거리	Aura-based	⬜ 미구현	1주
5 느리게	Significance/LOD	⬜ 미구현	3~5일 (+ 매니저 밑작업 1주)

5가지는 `enum` 하나로 런타임 전환 가능해야 한다. 그래야 같은 하네스로 연속 측정할 수 있다.

02. 측정 기준

0. 원칙

1. **재고 나서 고친다.** 먼저 적용해버리면 기준선이 흔들려 각 방법의 기여를 분리할 수 없다
2. **손으로 걷는 측정은 신뢰도가 0이다.** 경로가 매번 달라서 5가지를 비교할 수 없다
3. **5가지 전부 같은 하네스로 잰다.** 하나라도 다른 방식으로 재면 표 전체가 무효다
4. **평균 FPS는 쓰지 않는다.** 히칭이 평균에 묻힌다
5. ResourceSystem에서 세운 변수 통제·워밍업 규율을 그대로 따른다. 같은 덱 안에서 한쪽만 손측정이면 더 도드라진다

---

1. 하네스 — 만들어야 할 것 4개

1.1 스트레스 테스트 씬

현재 맵은 스포너 23개가 고정이라 **몬스터 수를 바꿀 수 없다.** 비교가 불가능하다.

- 평평한 바닥 + NavMesh만 있는 빈 씬
- 스포너를 **코드로 N개 배치**. 인스펙터에 `목표 몬스터 수`를 입력하면 그만큼 생성
- 배치는 **고정 시드 의사난수** — 매 실행 같은 배치여야 한다
- 밀도를 일정하게 유지하려면 N에 비례해 맵 크기도 키운다 (규모 실험의 전제)

> ⚠ **더미 프리팹 계열을 따로 둘 것.** 실제 몬스터 프리팹으로 3000개를 스폰하면 렌더·애니메이션에 파묻혀 AI 비용이 안 보인다. "무엇을 재는 실험인가"에 따라 갈아끼운다. - `AI만` — 렌더러 없음. 시뮬레이션 비용 측정용 - `풀 프리팹` — 실물. 종합 비용·메모리 측정용

1.2 자동 워크 하네스

플레이어를 웨이포인트로 스크립트 이동시키는 재현 시나리오. 60초 후 자동 종료.

1.3 방식 전환 스위치

```
enum SimMode { All, Sector, SectorWithCombatHold, DistanceBand, TickLod }
```

런타임에 갈아끼울 수 있어야 한다. 씬을 다시 만들면서 재면 조건이 어긋난다.

1.4 자동 로그 출력

60초가 끝나면 한 줄로 기록. 기존 자산 재사용: `MetricsHotkeys`(F5~F9) · `ResourceMetrics.LogRuntimeMemory` · `LogAliveReport`

---

2. 시나리오 3종 — 승자가 갈리는 축

N만 늘리면 방법4가 전 구간 승리라 표가 단조롭다. **승자를 가르는 것은 시나리오다.**

코드	이름	내용	무엇을 드러내나
`WALK`	종주	맵을 한쪽 끝에서 반대쪽까지 통과	진입 히치, 정상 상태 비용
`EDGE`	경계 왕복	구역 경계에서 전투하며 10회 왕복	**방법2/3을 가르는 유일한 지표.** 경계 소실·깜빡임
`PULL`	밀집	몬스터를 한곳으로 유인해 몰아넣음	**방법4가 지는 무대.** 방법5의 존재 이유

`EDGE`가 없으면 비교표의 correctness 행이 통째로 빈다. `PULL`이 없으면 방법4가 전 칸 승리로 나와서 "그냥 성능 측정"이 된다.

---

3. N 3단계

N	목적
200	현재 프로젝트 규모대. **5가지가 다 비슷하게 나올 것이다 — 그 사실 자체를 기록한다**
1000	갈라지기 시작하는 지점
3000	구조적 차이가 드러나는 지점

200 구간의 "구분되지 않음"을 당당하게 쓰는 것이 중요하다. 그래야 01_Problem 3.3("5개짜리 맵이라 안 터진 것이지 해결한 게 아니다")이 약점이 아니라 **관측 결과**가 된다.

---

4. 지표 정의

지표	정의	왜
p99 프레임타임	링버퍼 상위 1%	최악 구간. 체감을 지배한다
1% low	하위 1% FPS	업계 관행 지표
히칭 프레임 수	`>33ms` 프레임 개수	진입 히치를 직접 센다
평균 / 최대 활성 몬스터 수	시뮬레이션 중인 개체 수	방법3이 방법2보다 늘어나는 것을 잡는다
35m 내 몬스터 수 분포	최댓값·분포	**방법5 착수 판단의 근거**
디스폰 후 메모리 MB	`ResourceMetrics.LogRuntimeMemory`	G4 검증. 방법4의 조건부 이득
AI 시뮬 구간	`ProfilerMarker`	렌더 비용과 분리

평균 FPS는 기록하지 않는다.

---

5. 측정 절차

```
1. 개발 빌드로 실행 (에디터 아님)
   — ENABLE_GAME_LOG 심볼 확인. 없으면 로그가 스트립되어 파일이 안 생긴다
2. 워밍업 30초 — 셰이더 컴파일·풀 예열이 끝난 뒤부터 계측
3. 시나리오 재생 60초 (자동 워크)
4. 로그 한 줄 출력: mode, N, scenario, p99, 1%low, hitch, avgAlive, maxAlive, MB
5. 같은 조건 3회 반복 → 중앙값 채택
6. 다음 조건으로
```

5 방법 × 3 시나리오 × 3 N = 45 조합. 3회 반복이면 135회 × 60초 ≈ 2시간 15분. 자동화되어 있으면 반나절이면 끝난다. 자동화가 안 되어 있으면 절대 못 한다.

---

6. 측정 매트릭스 (채울 표)

시나리오별로 아래 표를 하나씩. 값은 `p99 / 히칭수 / 평균활성`.

N	방법1 전체	방법2 구역	방법3 구역+예외	방법4 거리	방법5 느리게
200					
1000					
3000					

별도로 한 번만 재는 항목:

항목	방법1	방법2	방법3	방법4	방법5
경계 소실 발생 여부 (`EDGE`)					
디스폰 후 메모리 MB					
35m 내 최대 몬스터 수 (`PULL`)					

---

7. 산출물

- `Baseline.md` — 방법2 기준선 수치 + 로그 원본 + **경계 소실 영상**
- `Results.md` — 45조합 표 + 03_Predictions와의 대조 결과
- 각 방법 구현 시점의 커밋 해시 — "내 v1 vs 내 v2" Before/After의 근거

03. 측정 전 예측 (박제)

> ⚠ **이 문서의 모든 내용은 아직 측정하지 않은 예상이다.** 재기 **전에** 써두는 것이 목적이다. 측정이 끝나면 04_Comparison에 실측을 채우고, 여기 적힌 예측이 맞았는지 **틀렸는지까지** 대조해서 기록한다. 예측을 나중에 고쳐 쓰면 이 문서는 아무 가치가 없다. **수정 금지, 추가만.**

왜 예측을 먼저 박제하나

측정 결과만 보여주면 "벤치마크 돌렸네"로 끝난다. **예측 → 측정 → 일치/불일치와 그 원인**이 있어야 구조를 이해하고 있다는 증거가 된다. 불일치가 나오면 더 좋다. 왜 틀렸는지 설명하는 것이 맞힌 것보다 강하다.

---

1. N에 대한 곡선 모양 예측

방법	예상 형태	근거
1 전부 켜기	**N에 비례.** 기울기 최대	관리가 없다
2 구역	**N에 비례. 기울기는 방법1과 같고 상수배(약 1/구역수)만 낮음**	구역 수가 고정이면 구역당 개체 수가 N에 비례
3 구역+예외	방법2와 **같은 기울기 + 유예분 오프셋**	구조는 방법2와 동일
4 거리	**거의 평평.** 활성 개체 수가 N과 무관	밀도가 일정하면 반경 내 개체 수만 좌우
5 느리게	**N에 비례하되 기울기가 크게 완만**	2Hz 버킷도 N에 비례해 늘어난다. 개체는 아무도 안 지운다

이 표의 핵심 한 줄

> **방법2는 상수배 개선이고, 방법4는 점근적 개선이다.** 구역을 10개로 쪼개도 방법2는 여전히 N에 비례한다. 맵이 커지면 다시 무너진다.

방법5는 **점근적 개선이 아니다.** 완만할 뿐 여전히 N에 비례한다. 이 구분이 "왜 4와 5를 겹치는가"의 근거다.

단서

방법4도 완전 상수는 아니다. CullingGroup이 스포너 수만큼 스피어를 돈다. 다만 개체당 AI 비용보다 2~3자릿수 싸서 평평해 보일 것으로 예상한다. **이 예상도 검증 대상이다.**

---

2. 시나리오별 승자 예측

시나리오	예상 승자	예상 패자	이유
`WALK` 종주	**방법4**	방법1	이동 중에는 반경 밖이 대부분
`EDGE` 경계 왕복	**방법3 · 방법5** (정확성 기준)	**방법2 (소실 발생)**	방법2만 correctness 실패
`PULL` 밀집	**방법5**	**방법4 (무용지물)**	전부 반경 안이라 거리로 걸러낼 게 없다

승자가 세 줄에서 다르게 나오는 것이 이 실험의 목적이다. **한 방법이 전 줄에서 이기면 비교가 아니라 그냥 성능 측정이다.**

---

3. N=200 구간 예측 ★

> **5가지가 유의미한 차이를 보이지 않을 것이다.**

현재 프로젝트 규모(스포너 23개)에서는 어떤 방법을 써도 비슷하게 나올 것으로 본다. 이것이 맞다면 **"5개짜리 맵이라 안 터진 것이지 해결한 것이 아니다"** 가 추측이 아니라 관측이 된다.

---

4. 틀릴 가능성이 높은 지점 (미리 적어둔다)

4.1 방법3이 방법2보다 나빠지지 않을 수 있다 ★

경계 전투는 상시가 아니다. 평균으로는 **방법2와 거의 같게** 나올 가능성이 상당하다.

- 그래도 확실히 참인 것: **성능이 좋아지지는 않는다** (버그만 고쳤다)
- 그래도 확실히 참인 것: **최악의 경우(경계에서 계속 전투) 방법1로 수렴한다**
- 따라서 서사는 "3번이 성능을 되돌린다"가 아니라 **"3번은 성능을 하나도 개선하지 못한다"** 에 걸어둔다

`EDGE` 시나리오를 공격적으로(경계에 오래 머물며 전투) 설계하면 차이가 드러날 수 있다. 드러나지 않으면 그대로 기록한다. **예측이 틀린 것도 결과다.**

4.2 방법5의 이득이 예상보다 작을 수 있다

BT 평가 비용이 전체에서 차지하는 비중이 작으면(렌더·애니메이션이 지배적이면) 틱을 깎아도 프레임이 안 좋아진다. 그래서 `AI만` **더미 프리팹과** `풀 프리팹` **둘 다로 재야** 원인을 분리할 수 있다.

4.3 방법4의 팝인이 생각보다 거슬릴 수 있다

밴드 25m가 화면과 거의 같으면 화면 가장자리에서 스폰이 보인다. 밴드를 넓히면 이득이 줄어든다. **이 트레이드오프의 실제 임계점은 재봐야 안다.**

4.4 메모리가 방법4에서도 안 줄 수 있다

풀·핸들 반납 정책을 같이 고치지 않으면 **디스폰해도 MB가 그대로다**(G4). 방법4 단독 측정에서 메모리가 안 줄면 그것이 정상이며, 별도 작업이 필요하다는 증거다.

---

5. 검증 후 기록 규칙

측정이 끝나면 04_Comparison에 아래 형식으로 남긴다.

```
예측: 방법2는 N에 비례, 기울기는 방법1과 동일
실측: (수치)
판정: 일치 / 불일치
불일치라면 원인:
```

예측이 틀린 항목을 지우지 않는다. 틀린 예측과 그 원인 분석이 이 실험의 가장 강한 산출물이다.

04. 비교표

> ⚠ **현재 아래 정성 비교는 전부 "측정 전 예상"이다.** 실측이 들어오면 이 배너를 지우고 `(실측)` 표기로 바꾼다. 정량 표는 비어 있으며 02_Measurement의 절차대로 채운다.

1. 정량 표 (측정 후 채움)

1.1 `WALK` — 종주

값 형식: `p99(ms) / 히칭수 / 평균활성`

N	방법1 전체	방법2 구역	방법3 구역+예외	방법4 거리	방법5 느리게
200					
1000					
3000					

1.2 `EDGE` — 경계 왕복

N	방법1	방법2	방법3	방법4	방법5
200					
1000					
3000					

1.3 `PULL` — 밀집

N	방법1	방법2	방법3	방법4	방법5
200					
1000					
3000					

1.4 한 번만 재는 항목

항목	방법1	방법2	방법3	방법4	방법5
경계 소실 발생 (`EDGE`)					
디스폰 후 메모리 MB					
35m 내 최대 몬스터 수 (`PULL`)					

2. 정성 비교 — 측정 전 예상

	1 전부 켜기	2 구역	3 구역+예외	4 거리	5 느리게
분류	—	Zone-based	Hybrid	Aura-based	Significance/LOD
축	—	존재	존재	존재	**품질**
경계에서 몬스터 소실	없음	**있음**	없음	없음	없음
팝인	없음	심함	심함	있음	**없음**
종주 시 가벼움	나쁨	보통	보통	**최고**	좋음
밀집 시	나쁨	못 거름	못 거름	**무용지물**	**유일하게 효과**
맵이 10배 커지면	무너짐	다시 무너짐	다시 무너짐	**영향 없음**	완만해지지만 계속 늘어남
메모리 감소	안 줌	안 줌	안 줌	**줄일 수 있음** (조건부)	안 줌
구현 난이도	없음	쉬움	보통	보통	어려움

표 읽는 법

칸마다 이기는 방법이 다르다. 한 방법이 전 칸 승리면 그건 비교가 아니라 성능 측정이다.

특히 마지막 두 줄이 결론을 만든다:

> 방법4는 **규모**를 해결하지만 **밀집**에 무력하고, 방법5는 **밀집**을 해결하지만 **규모**에 약하다. 그래서 둘 중 하나를 고르는 것이 아니라 **층을 나눠 겹친다.**

조건부 표기 주의

- **방법4 "메모리 줄일 수 있음"** — 풀·핸들 반납 정책을 같이 고쳐야 실현된다. 방법4 단독으로는 G4 그대로다
- **방법5 "완만해지지만 계속 늘어남"** — 개체를 지우지 않으므로 점근적으로는 여전히 O(N). 방법4와 달리 확장성 문제를 해결하지 못한다

---

3. 판정 입자 스펙트럼

```
[존재 축]
방법1          방법2            방법3              방법4
맵 전체   →   구역 100m   →   구역 + 예외   →   반경 약 45m
(∞)                                              ↑ 화면 지면 ~25m

[품질 축] ← 직교
방법5   끄지 않고 틱 등급만 강등  →  방법4와 동시 적용 가능
```

방법1~4는 임의의 후보가 아니라 **한 축 위의 네 점**이다. 이걸 먼저 보여주면 비교표가 "그때그때 떠올린 아이디어 모음"이 아니라 **하나의 설계 공간을 훑은 결과**로 읽힌다.

---

4. 예측 대조 기록 (측정 후)

03_Predictions의 각 항목에 대해:

```
예측:
실측:
판정: 일치 / 불일치
불일치 원인:
```

틀린 예측을 지우지 않는다. 틀린 예측과 그 원인 분석이 가장 강한 산출물이다.

---

5. 결론 문장 (측정 후 확정)

측정이 끝나면 아래 빈칸을 채워 한 문단으로 만든다.

> 표를 보면 상황마다 승자가 다르다. 그런데 이 게임은 **화면 25m · 연속 필드 · 경계 전투 빈발**이라 `WALK` 와 `EDGE` 가 지배적이다. 따라서 **방법4를 채택**하고 경계 깜빡임은 히스테리시스로 막았다. `PULL` (밀집)은 실측 분포상 35m 내 최대 ___마리로 ___ 판단하여 방법5를 ___ 했다. 메모리는 방법4 + 풀 반납 정책으로 ___MB → ___MB.

조건이 달랐다면 답도 달랐다는 것을 **한 번만** 덧붙인다 — 구역이 화면보다 작고 포털로 분리된 실내 던전이면 방법2가 여전히 최선이고, 몬스터가 수십 마리 규모면 방법1이 정답이다.

05. 포트폴리오 구성

0. 이 파트가 파는 것

기법이 아니라 **선택의 근거**다.

컬링·LOD·풀링은 신입 포트폴리오에 널려 있다. "적용했습니다"는 변별력이 0이다. 드문 것은 **여러 후보를 실제로 만들어서 같은 기준으로 재고, 자기 게임 조건에 대입해 고른 기록**이다.

---

1. 서사 골격

```
① 버그 영상 — 때리던 몬스터가 눈앞에서 사라진다          ← 여기서 시작한다
      ↓
② 원인 — 판정 단위 100m vs 화면 25m (실측)
      ↓
③ 1차 시도 — 싸우는 대상은 예외 (방법3)
      → 버그는 고쳐졌는데 성능은 조금도 안 나아짐          ← 전환점
      ↓
④ 틀 교체 — 거리 기반 (방법4)                          ← 본론
      → N 스윕 + 시나리오 비교표                        ← 증거
      ↓
⑤ 방법4가 못 하는 칸 — 밀집 → 방법5는 축이 다름          ← 확장
      ↓
⑥ 그런데 껐는데 메모리가 안 줄었다 (G4)                 ← 자기 감사 → 리소스 파트로
```

방법1(전부 켜기)은 ②에 **한 줄로 흡수**된다. 5가지 비교표는 ④의 근거로 **한 장** 들어간다. 세트는 5개인데 이야기는 하나로 흐른다.

왜 방법1이 아니라 방법2에서 시작하나

방법1은 내가 설계한 것이 아니라 **"아직 아무것도 안 한 상태"** 다. 거기서 시작하면 Before가 인위적이라 "그 Before가 실제 빌드 구성이었나요?" 한 번에 무게가 빠진다. 방법2는 **내가 만들어서 굴리던 것**이라 Before가 정직하다. = "내 v1 커밋 vs 내 v2 커밋".

왜 분류표가 아니라 버그로 시작하나

포트폴리오 수십 개를 훑는 사람이 멈추는 지점은 "엔티티 관리 5가지 방법 비교"가 아니라 **"때리던 몬스터가 눈앞에서 사라졌습니다"** 다. 분류·이론은 그다음에 나와야 한다.

---

2. 비중 배분 — 균등하게 주지 않는다

지면은 곧 "이게 그만큼 중요하다"는 신호다. 5개를 똑같이 한 장씩 주면 **다 얕다**는 뜻이 된다.

방법	지면	역할
1 전부 켜기	**반 줄**	기준선. "안 하면 이렇다"
2 구역	**1장**	출발점. **버그 영상**이 여기서 나온다
3 구역+예외	**1장**	전환점. 왜 틀을 바꿔야 했는가
4 거리	**2~3장**	본론. 실제로 만든 것. 구현 판단이 전부 여기
5 느리게	**1장**	축이 다르다 + 겹쳐 쓴다는 결론
비교표	**1장**	결론이 아니라 **근거**
메모리 마무리	**1장**	자기 감사 → 리소스 파트 연결

---

3. 이론은 라벨로, 깊이는 구현 판단으로 ★

이게 가장 중요한 구분이다.

	쓰는 법	예
이론 (라벨)	한 줄. 읽은 것을 증명	"방법2는 Zone-based, 방법4는 Aura-based로 분류됩니다"
깊이 (구현 판단)	여기에 지면을 쓴다. **만들어본 사람만 아는 것**	아래 참조

깊이로 쓸 것 — 방법4의 구현 판단 4가지:

1. **몬스터가 아니라 스포너를 등록한 이유** — 스포너는 안 움직이므로 스피어 갱신 비용 0
2. **기준점을 카메라가 아니라 플레이어로 둔 이유** — 카메라 회전만으로 세계가 스폰되면 안 된다
3. **들어올 때와 나갈 때 기준을 다르게 준 이유** — 히스테리시스. 없으면 경계에서 깜빡인다
4. **디스폰돼도 리스폰 타이머는 스포너가 갖는 이유** — 화면 밖 세계가 멈추지 않게

이론 5개보다 **구현 판단 4개가 훨씬 세다.** Zone-based/Aura-based를 자세히 설명할수록 "서베이 논문 읽은 학생"으로 읽힌다.

---

4. 덱 안에서의 위치 — 중복 회피

이터널리턴(DX11) 파트에 이미 **QuadTree(충돌 135배·피킹 4.6배)** 와 **Spatial Grid(21.5배)** 가 있다. 이 파트가 "또 공간 분할로 몇 배"로 끝나면 **엔진 기능으로 다시 한 하위호환**으로 읽힌다.

파트	축	증명하는 것
이터널리턴 QuadTree	알고리즘 최적화	덜 계산하는 법
이 파트	**시스템 설계 판단 + 자기 코드 감사**	**무엇을 존재시킬지 정책을 정하는 법**

축을 확실히 가르려면 **끝을 성능이 아니라 메모리로 닫는다.**

> "공간 분할했다고 말해온 것의 실체는 CPU 절감뿐이고, 메모리는 1바이트도 줄지 않았다" (G4)

이 자기 감사 문장이 덱 전체에서 유일무이하다. 남이 못 쓰는 문장이다. 그리고 그대로 Addressable 리소스 파트로 이어진다.

- ❌ "거리 기반이 제일 빨랐습니다" → 그냥 성능 슬라이드
- ⭕ "최적화라고 불러온 것이 절반이었고, 나머지 절반은 리소스 수명이었다" → 다른 종류의 슬라이드

---

5. 함정 — 반드시 피할 것

5.1 안 잰 숫자를 단정하지 않는다 ★

"1000마리가 넘어가면 프레임이 떨어집니다" 같은 문장은 **측정 전에는 쓰지 않는다.** "이 숫자 어디서 나왔어요?" 한 번에 문서 전체 신뢰가 무너진다. 쓰려면 `(예상 – 측정 전)`을 명시한다.

5.2 "상황에 따라 다릅니다"로 끝내지 않는다

비교표를 다 보여준 뒤 반드시 오는 질문이 **"그래서 뭘 쓰셨어요?"** 다. 여기서 상황론으로 답하면 판단을 회피한 사람이 된다. 표는 근거이고 **결론은 하나여야 한다.**

5.3 "기획 상"이라는 말을 쓰지 않는다

"기획 상 이게 나았다"는 **기획자가 정해준 걸 따랐다**로 들린다. 판단 근거는 취향이 아니라 **측정 가능한 게임 조건**이다.

> "화면에 보이는 땅이 25m인데 구역은 100m였고, 연속 필드라 경계 전투가 자주 일어납니다. 이 조건에서 4번이 유리했습니다."

5.4 결론을 "4번이 제일 좋았다"로 쓰지 않는다

방법4와 5는 축이 직교해서 경쟁 관계가 아니다. 하나만 고르면 결론의 가장 좋은 부분을 버리는 것이다.

> **4번으로 누가 존재할지를 정하고, 5번으로 존재하는 것들을 얼마나 자주 돌릴지를 정했습니다.**

5.5 방법3을 "탈락한 안"으로 쓰지 않는다

방법3은 **틀을 바꿔야 했던 이유의 증거**다. "3번을 해보니 섹터 안에서는 두 마리 토끼를 못 잡는다는 게 나와서 4번으로 갔다" — 이 연결이 있어야 방법4가 "하나 더 해봤다"가 아니라 필연이 된다.

5.6 만들지 않은 것을 비교표에 올리지 않는다

"3가지를 비교했습니다"라고 쓴 포트폴리오의 상당수가 실제로는 하나만 구현했다. 그래서 **"2번 방법은 실제로 만들어보셨어요?"** 가 온다. 아니라고 하면 표 전체가 의견으로 격하된다.

- 표에 올린 방법은 전부 실제로 돌아가야 한다 (`enum` 하나로 전환)
- 각 칸의 숫자는 같은 하네스로 잰 것이어야 한다
- 구현하지 않은 안(인접 섹터 켜기)은 표가 아니라 **각주**로

5.7 문헌을 과대 인용하지 않는다

Interest Management 문헌은 **MMO 네트워크 트래픽** 절감이 목적이고, 이 작업은 **클라 CPU/메모리**다. 개념은 옮겨오지만 비용 모델이 다르다. **이 구분을 먼저 말하고** 인용한다. 아는 면접관이면 반드시 찌르는 지점이고, 먼저 구분하면 오히려 점수가 된다.

5.8 인용 순서를 뒤집지 않는다

- ❌ "Mass AI 1단계가 0~25m라서 저도 25m로 했습니다" — 약하다
- ⭕ "제 카메라 실측이 25m였고, 나중에 보니 Mass AI 1단계도 0~25m더라" — 강하다

먼저 재고 나중에 대조한 순서를 유지한다.

---

6. 경쟁력에 대한 솔직한 평가

흐름이 경쟁력을 결정하지 않는다. 실제로 몇 개를 만들어서 쟀는지가 결정한다.

상황	결과
2개(현재)만 있고 나머지는 설명	어떤 흐름을 짜도 **기술 조사 보고서**
3개(2→3→4)를 만들어 같은 하네스로 측정	흐름이 엉성해도 **충분히 경쟁력 있음**
5개 전부 깊게	과함. 방법1은 스위치 하나, 방법5는 "다음 단계"로 남겨도 된다

즉 고민할 것은 구성이 아니라 **"방법3과 4를 만들 시간이 있는가"** 다. 있으면 이 주제는 살아나고, 없으면 어떤 구성으로도 안 살아난다.

---

7. 작업 순서

```
1. 경계 소실 Before 영상 촬영        ← 지금 당장. 고치면 다시 못 찍는다
2. 측정 하네스 (테스트 씬 + 자동 워크 + enum 전환 + 로그)
3. 방법3 구현 → 측정
4. 방법4 구현 → 측정
5. 방법5 구현 → 측정
6. 45조합 측정 → 04_Comparison 표 채우기 → 03_Predictions 대조
7. 풀·핸들 반납 정책 → 메모리 전후 측정 → 리소스 파트 연결
```

06. 참고 자료

> 인용은 **근거로만** 쓴다. 자랑거리로 쓰면 "읽었구나"로 끝나고, 근거로 쓰면 "내 판단이 업계 분류와 맞물린다"가 된다.

---

1. 이 문제의 정식 이름 — Interest Management / AOI

Interest Management for Distributed Virtual Environments: A Survey ACM Computing Surveys, Vol 46 No 4 — https://dl.acm.org/doi/10.1145/2535417

Area of Interest Management in Massively Multiplayer Online Games Springer — https://link.springer.com/rwe/10.1007/978-3-319-08234-9_239-1

무엇에 쓰나

5가지 방법에 정식 분류명을 붙인다. "제가 5가지 생각해봤습니다"가 **"업계·학계 분류를 내 게임 조건에서 실측 비교했습니다"** 로 격상된다.

학술 분류	대응	비고
Zone-based	방법2	세계를 구역으로 쪼개고 속한 구역의 갱신만 받음
Aura-based	방법4	참여자 주변 "아우라"로 관심 범위 표현
Visibility-based	(미사용)	실제 가시성으로 필터링
Hybrid	방법3	zone과 aura를 섞음 (MASSIVE-3 등)

서베이가 지적한 Zone-based의 약점 = 우리 G2

> 구역 크기를 적절히 고르지 못하면 불필요한 데이터가 여전히 전달된다

화면 25m에 섹터 100m. **실측으로 발견한 것을 서베이가 일반 명제로 써둔 셈**이다.

⚠ 인용 시 반드시 먼저 말할 것

이 문헌은 **MMO 네트워크 트래픽** 절감이 목적이고, 이 작업은 **클라이언트 CPU/메모리**가 목적이다. 개념(누구를 신경 쓸 것인가)은 그대로 옮겨오지만 **비용 모델이 다르다.** 먼저 구분하고 인용하면 오히려 점수가 된다. 안 하면 아는 면접관에게 찔린다.

---

2. 방법5의 원형 — UE Significance Manager

Significance Manager in Unreal Engine (Epic 공식 문서) https://dev.epicgames.com/documentation/en-us/unreal-engine/significance-manager-in-unreal-engine

오브젝트별 "중요도" 점수를 계산하고 틱 주기·LOD·이펙트 여부를 조절하는 프레임워크. 타입별 예산도 있다 — 카메라 근처에 플레이어가 몰리면 고품질로 도는 AI 폰 수를 그만큼 깎는 식.

결정적 인용 ★

> 트리거 볼륨에 의존할 수 없는 비선형 경험에서 특히 유용하다

이것이 방법2 → 방법4 전환의 공식 문서 근거다. 트리거 볼륨(Sector)에서 벗어나려는 이유를 엔진 개발사가 그대로 써두었다.

---

3. 현업의 답 — UE5 Mass AI 크라우드 LOD

Building Crowds and Traffic in UE5: Mass AI and 10,000 NPCs at 60fps — StraySpark
https://www.strayspark.studio/blog/crowd-traffic-simulation-ue5-mass-ai

거리 기반 4티어. **누구도 디스폰되지 않는다.**

티어	거리	내용
1	**0~25m**	스켈레탈 메시, 본 애니메이션, 물리, 풀 AI
2	25~60m	VAT 정적 메시, 물리 없음, 경로 추종만
3	60~150m	인스턴싱(드로우콜 1), 동일 포즈, 회피 없음
4	150m+	임포스터/파티클

티어1이 티어3보다 약 50배 비싸고, 10,000 에이전트가 4.6ms(RTX 4080). 전환 완화는 **디더링 · 크로스페이드 · 시간 오프셋 분산**(보이는 "파도" 방지).

⚠ 인용 시 주의

50배 / 4.6ms는 Epic 공식 수치가 아니라 한 스튜디오 블로그의 벤치마크다. 출처를 그대로 밝힐 것.

우리 설계와 대조되는 점

- 티어1 **0~25m** ↔ 우리 화면 지면 ~25m → **숫자 감각이 맞다** (단, 먼저 재고 나중에 대조한 순서 유지)
- 시간 오프셋 분산 ↔ 우리 위상 오프셋 → 같은 대응
- 디더링/크로스페이드 → 우리 계획엔 히스테리시스만 있음. **팝인 완화 수단이 하나 더 있다**

---

4. 방법4의 구현 수단 — Unity CullingGroup

CullingGroup API (Unity 공식 매뉴얼) https://docs.unity3d.com/Manual/CullingGroupAPI.html

- 바운딩 **스피어만** 지원 (성능상 다른 형태 없음)
- 기준점과 각 스피어 최근접점 사이 거리를 **밴드(정수)로 양자화**
- 마지막 밴드 밖은 invisible 처리 → 완전 비활성화 구현에 그대로 쓸 수 있음
- ⚠ **카메라 컬링 시점에만 갱신된다.** 비동기적으로 다루어야 한다 (구현 시 함정)

---

5. 프로젝트 내부 참조

문서	무엇
../01_Problem.md	G1 경계 소실 · G2 판정 단위 · G3 스폰 몰림 · G4 메모리
../03_Design.md	A층(존재)/B층(품질) 분리 원칙 — 방법4/방법5에 대응
../05_Roadmap.md	단계별 일정. 본 트랙의 측정 계획이 S0를 확장한 것
../../ResourceSystem/	변수 통제·개발빌드 계측의 선례. 같은 급으로 맞춘다
../../AbilitySystem/STATUS.md	측정 규율의 다른 사례 (GC 측정 철회 판단 포함)
../../TroubleShooting/MemoryLeak-MonsterHPBar/	방법3의 "사용 중" 집합이 새면 같은 계열 사고

---

6. 독립적으로 도달한 것 (검증됨)

조사 결과, 아래는 문헌을 보기 전에 ../03_Design.md에서 이미 내린 판단이며 업계 구조와 일치한다. **그대로 밀어도 된다.**

- **A층(존재) / B층(품질) 분리** — Mass AI(품질)와 AOI(존재)가 실제로 별개 시스템이다. *"A는 명단을 정하고, B는 그 명단 안에서 자원을 배분한다"*
- **밴드 25/35/50m** — Mass AI 티어1이 0~25m
- **위상 오프셋** — Mass AI도 시간 오프셋 분산으로 "파도"를 막는다
- **B층(점수·예산)을 조건부로 미룬 것** — Significance Manager 문서도 예산·정원을 고급 사용례로 분류